Frontend Design Specification

Passive threat detection console — React interface

Smart India Hackathon · design and build brief for the frontend team · v1

1. What this interface is

The backend watches a one-directional copy of network traffic and produces scored alerts. It cannot send anything back into the network. It cannot block, quarantine, reset a connection, or query a host. It can only observe and report.

That constraint is the design thesis. **This is an instrument, not a console.** Every screen is a readout. There are no action buttons on alerts, because there are no actions — and that absence is deliberate, visible, and worth pointing out during the demo. A judge who notices there is no "Block" button has understood the architecture.

1.1 What the interface has to prove

Four of the graded requirements can only be demonstrated through this UI. Design decisions should be traced back to them.

Requirement	What the UI must show
Streaming, not batch	Visible, continuous liveness. Not a number that updates — motion that never stops while traffic flows.
Supporting evidence per alert	For any alert, the exact features, their values, and the thresholds they crossed. This is the highest-value screen in the product.
Stated throughput	Live flows per second, megabits per second, and packet rate, always on screen.
Confidence and severity	Both present on every alert, visually distinct from each other, and never conflated.

1.2 Who is looking at it

Judges, standing behind someone, for five to ten minutes, in a room with poor lighting and a projector. They are technical but have not read your documentation. The interface has roughly eight seconds to communicate "this system is detecting real threats right now" before attention moves on.

Design for that, not for an eight-hour analyst shift. Density is good; explanation-free legibility is better.

> **EXPLICITLY OUT OF SCOPE**
> No sign-in screen, no sign-up flow, no user management, no settings pages, no onboarding. The application opens directly into the live view. If authentication is ever needed it is a single gate in front of the whole app, built last, and never the first thing anyone sees. Time spent on auth is time not spent on the screens that are actually graded.

2. Design direction

2.1 The reference point is instrumentation, not dashboards

The obvious approach — dark navy panels, neon cyan accents, translucent cards, a row of large KPI numbers — is what most competing teams will build, because it is what every dashboard template produces. It also fights the subject: those visuals suggest control, and this system controls nothing.

Draw instead from measurement instruments: oscilloscopes, spectrum analysers, seismographs, chart recorders. Precise gridlines. Tick marks with calibration values. Fixed-decimal readouts that do not jitter in width as numbers change. Restraint everywhere except where a signal appears.

> **THE SINGLE MOST IMPORTANT RULE**
> Colour is reserved entirely for severity and threat class. Nothing else on screen is saturated — not headers, not borders, not buttons, not icons, not charts of benign traffic. When an alert appears, its colour is the only vivid thing on the display, so it demands attention without needing to glow, pulse, or animate. Colour carries information; it never decorates. This single constraint will do more for the look of this product than any other decision in this document.

2.2 Colour tokens

Surface and structure — the entire non-alert interface

Token	Value	Use
`--bg`	`#0B0D0F`	Application background
`--panel`	`#14181B`	Panel surfaces
`--panel-2`	`#1B2024`	Raised surfaces, selected row
`--rule`	`#252C31`	Borders, dividers, chart gridlines
`--rule-bright`	`#38424A`	Axis lines, active borders
`--text`	`#E9ECED`	Primary text, readout values
`--text-2`	`#8B959C`	Labels, units, secondary data
`--text-3`	`#5A646B`	Axis ticks, disabled, placeholder

Severity ramp — the only saturated colours in the product

Token	Value	Use
`--sev-low`	`#4E9C7F`	Low severity
`--sev-med`	`#D2A03E`	Medium severity
`--sev-high`	`#DB7038`	High severity
`--sev-crit`	`#C8453D`	Critical severity
`--baseline`	`#41718F`	Learned-normal bands and reference traces only. Never an alert colour.

Severity is applied as a 2px left border on alert rows, as the colour of the alert's mark on the Wire, and as a small filled square in dense listings. It is never used as a full-panel background — a wall of red destroys the discipline that makes the ramp work.

> **SEVERITY IS NOT CONFIDENCE**
> These are different quantities and must never share an encoding. Severity is colour. Confidence is a numeric readout plus a thin horizontal bar. An alert can be critical severity at 0.62 confidence, and the interface has to make that combination readable at a glance. If a designer conflates them, the product misrepresents what the backend actually produces.

Threat class marks

Eight detectors need to be distinguishable in dense listings. Do not give each a colour — that would break the severity discipline and eight hues is beyond what anyone can learn. Use a two-letter monospace code in a bordered box, at `--text-2`:

```
DDoS flood        DF        DGA domain        DG
Amplification     AM        DNS tunnel        DT
Port scan         PS        Encrypted C2      EC
Beaconing         BC        Exfiltration      EX
```

These codes are also what appears on the Wire, so they are learned within a minute of watching.

2.3 Typography

Two faces from one family, so they share proportions and metrics while doing different jobs.

Face	Role	Where
IBM Plex Mono	Data	Every value that came from the network: IP addresses, ports, timestamps, hashes, byte counts, domains, confidence numbers, feature values. Anything a machine produced.
IBM Plex Sans	Interface	Labels, headings, navigation, descriptions, empty states. Anything a human wrote.

The split is a rule, not a suggestion, and it does real work: a reader can tell at a glance which parts of the screen are observed data and which are interface chrome. IBM Plex was designed for technical systems interfaces, which is exactly this brief, and it avoids the Inter and Roboto defaults that make dashboards interchangeable.

Type scale

```
readout-lg   28px / 1.0  / Plex Mono 500 / tabular   throughput figures
readout      18px / 1.1  / Plex Mono 500 / tabular   confidence, key values
data         13px / 1.4  / Plex Mono 400 / tabular   alert rows, feature values
data-sm      11px / 1.35 / Plex Mono 400 / tabular   axis ticks, timestamps
heading      15px / 1.3  / Plex Sans 600             panel titles
body         13px / 1.5  / Plex Sans 400             descriptions, analyst text
label        10px / 1.2  / Plex Sans 600 / +0.08em uppercase   field labels
```

> **TABULAR FIGURES ARE MANDATORY**
> Every number in this interface updates live. Set `font-variant-numeric: tabular-nums` globally on mono text. Without it, digits change width as values change and the entire display shivers — which looks broken, and on a projector looks catastrophic.

2.4 Spacing and geometry

```
Space scale (px):   4  8  12  16  24  32  48
Border radius:      2px on interactive elements, 0 everywhere else
Border width:       1px rules, 2px severity indicators
Panel padding:      16px
Row height:         32px in dense listings, 40px in the alert stream
```

Near-zero radius is a deliberate choice: instruments have square bezels, and rounded cards read as consumer software. Keep 2px on buttons and inputs only, so interactive elements are distinguishable from readouts by shape alone.

2.5 Motion

Motion is used for exactly two things: showing that data is flowing, and showing that something new has arrived. Nothing else moves.

Event	Treatment
Wire advancing	Continuous, constant velocity, canvas-rendered. Never eases, never pauses.
New alert arrives	Row fades in over 140ms and its severity border draws from top to bottom. No slide, no bounce.
Panel or view change	Instant. Cross-fades on a live monitoring tool make it feel sluggish.
Hover	Background shifts to `--panel-2`. No scale, no shadow, no lift.
Value updates	No transition. A readout that animates between numbers is lying about when the value changed.

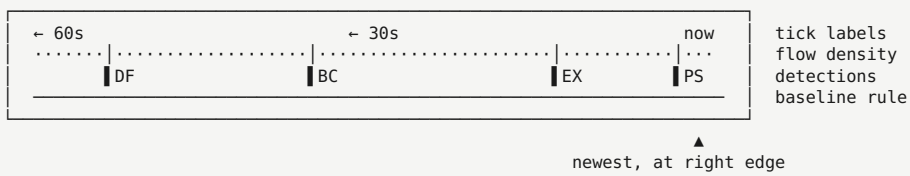
Under `prefers-reduced-motion`: the Wire redraws in one-second steps instead of continuously, and alert rows appear without fade. Everything remains fully functional.

3. The signature element: the Wire

A single horizontal trace, fixed across the top of every screen, showing traffic flowing past from right to left. Detections appear as marks on the trace at the position corresponding to when they were found.

It earns its place for a specific reason. Liveness is the hardest property to convey and the one requirement judges are most sceptical of — a number that increments could be a timer. A continuous trace that never stops while traffic flows, with detections landing on it in real time, is direct visual evidence that the system is streaming. It is also the clearest possible expression of the thesis: traffic moves one way, past an observer that can only mark what it sees.

3.1 Anatomy



- **Density trace** — a thin line whose height encodes flows per second at that moment. Drawn in `--text-3`. This is the ambient signal.
- **Detection marks** — a 2px vertical tick in the severity colour, with the two-letter threat code beside it. Marks persist as they scroll left.
- **Time axis** — sixty seconds of history, ticks every fifteen seconds, right edge is now.
- **Interaction** — hovering a mark shows a compact tooltip; clicking selects that alert in the stream below.

3.2 Implementation notes

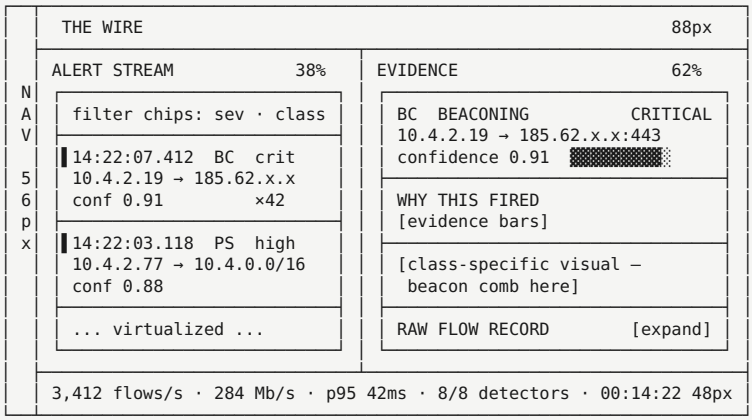
- Canvas, not SVG or DOM. At sixty frames per second with dozens of marks, DOM will not hold up and will drag the rest of the interface down with it.
- One `requestAnimationFrame` loop owns the Wire. It reads from a ring buffer that the WebSocket handler writes to. React state is never involved in the animation path.
- Fixed height of 88px. It is always visible, on every screen, and never scrolls away.
- When the replay is paused or the feed disconnects, the trace freezes and a `--text-3` label reads `feed stopped`. It never fakes movement.

4. Screen inventory and layout

Five views, one docked panel. Navigation is a persistent left rail, 56px wide, icon-plus-label, no submenus.

View	Priority	Purpose
Live	Critical	The demo screen. Alert stream and evidence side by side. This is what is on the projector for most of the presentation.
Incidents	High	Correlated alerts grouped per host, ordered along the kill chain.
Host	Medium	Single-host investigation. Timeline, baseline comparison, every alert involving it.
System	High	Throughput, latency distribution, per-detector state, and the read-only constraint proof.
Replay	Medium	Demo control. Load a capture, set replay speed, start and stop.
Analyst	Medium	Docked right panel, toggled from anywhere. Plain-language explanation and question answering.

4.1 Live view



Evidence gets the larger pane because it is the graded requirement and the thing that convinces. The alert stream is a selector; the evidence panel is the product.

Selecting an alert is instant — no loading state, no spinner. Everything needed is already in the alert payload. If a spinner appears here, the API contract is wrong.

4.2 Instrument bar

Fixed at the bottom of every view, 48px, monospace, tabular. Six readouts separated by thin vertical rules:

```
flows/sec   ·   Mb/s   ·   p95 latency   ·   detectors online   ·   uptime   ·   feed state
```

Values update at 1Hz — fast enough to feel live, slow enough to be readable. Feed state is the only element in the bar permitted colour, and only when disconnected.

5. Evidence: the component that wins

Every alert answers "why did this fire" in two layers: a universal evidence readout, then a visual specific to the threat class. The second layer is where this interface stops looking like a template.

5.1 Evidence bars (universal)

Every alert carries an evidence array of feature names, observed values, and the thresholds they crossed. Render each as a horizontal readout showing how far past the line the value landed.

```
interval_cv           0.041                                  CRITICAL
  0                    ├──────────●                  0.15|         0.5
                                  ▲ observed          threshold

observed_periods       42
  0                    ├──────────────────────●   |              50
                                             8 | threshold

mean_interval_s       60.2  ± 2.4
```

- Bar track in `--rule`, filled portion in `--text-2`, threshold as a 1px `--rule-bright` vertical rule with its value labelled.
- The observed marker takes the severity colour only when the value is on the wrong side of the threshold.
- Features with no threshold (contextual values) render as a plain label-value pair with no bar. Do not invent a scale to make them look uniform.
- Order by contribution, most decisive first.

5.2 Class-specific evidence visuals

Eight detectors, eight ways of being right. Each gets a visual that shows the detector's actual logic. These are the components worth spending design time on.

BC — BEACONING

The comb

Plot every connection in this tuple as a vertical tick on a horizontal time axis. A beacon renders as an evenly spaced comb. Human-driven traffic renders as scattered clusters. No legend or explanation is required — the difference is visible instantly, and it is the most convincing single image in the entire product.

```
this flow    | | | | | | | | | | | | | | | | | | | | | | | |   ← beacon
             └─────────────────────────────────────────────────┘
typical host |  ||   |      ||| |        |   ||  |       ← human
             └─────────────────────────────────────────────────┘
             0                                      60 min
```

Pair it with a small interval histogram: a beacon's intervals pile into one narrow bin, and the tightness of that pile is the detection. Show the measured jitter percentage as a readout beside it.

PS — PORT SCANNING

Fan-out matrix

A grid with destination port on the horizontal axis and time on the vertical, cells filled where the source made contact. A vertical scan fills a dense vertical band; a horizontal sweep fills across. The pattern names itself.

Fill cells in `--text-2`, and cells that were refused or reset in the severity colour — a wall of rejected connections is what makes a scan a scan.

DF · AM — FLOODING AND AMPLIFICATION

Entropy against rate

Two stacked traces sharing a time axis: packet rate above, source-IP entropy below, with the learned normal band drawn in `--baseline`. A spoofed flood shows rate climbing while entropy jumps out of the band. A direct flood shows rate climbing while entropy collapses. Both signatures are legible in one image, and the difference between them is exactly what the detector distinguishes.

For amplification, add a single readout showing response-to-request byte ratio with the reflector port labelled.

DG — DGA DOMAINS

The string inspector

Render the queried domain at `readout-lg` in mono, with an n-gram score heat drawn as a background band behind each character span — darker where the sequence is improbable against the legitimate-domain corpus. The suspicious region of the string is visible directly.

Below it, the host's NXDOMAIN sparkline over the last hour. A burst of failed resolutions is the behavioural half of the detection and it belongs on the same screen as the string half.

DT — DNS TUNNELLING

Subdomain fan-out

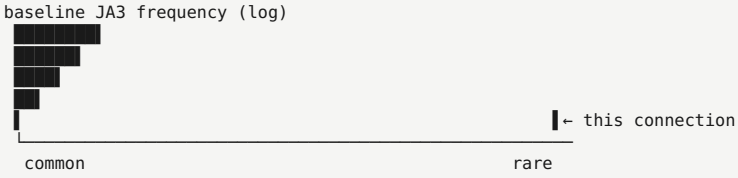
The parent domain as a fixed header, with a live-scrolling list of the distinct subdomains queried beneath it and a large cardinality counter. Legitimate domains resolve a handful of names; a tunnel generates hundreds of unique ones. The counter climbing while the list scrolls is the story.

Add a query-length distribution strip — tunnels sit far to the right of normal DNS.

EC — ENCRYPTED-SESSION MALWARE

Fingerprint rarity

A horizontal frequency distribution of every JA3 fingerprint seen during the baseline period, log-scaled, with this connection's fingerprint marked in the severity colour far out in the tail. The point — this client looks like almost nothing else on the network — needs no caption.



Beneath it, a short history of the fingerprints this host has previously presented. A machine that showed a browser fingerprint all week and suddenly presents a scripting-library one is the actual finding.

EX — EXFILTRATION

Baseline departure

Outbound bytes per hour for this host over the last several days, with the learned normal band drawn in `--baseline` and the current period breaking out above it in the severity colour. Beside it, two readouts: outbound-to-inbound ratio, and whether this destination has ever been contacted before.

Destination novelty deserves prominence. Volume alone produces false positives on backups; volume to somewhere never seen before is the real signal.

> **BUILD ORDER FOR THESE EIGHT**
> Build the comb first — it is the most convincing and the least work. Then the fan-out matrix, then entropy-against-rate. Those three cover the most demo-visible detectors. The DGA string inspector is fourth because it looks impressive and reads instantly. The remaining four can render as evidence bars only if time runs short; a missing class-specific visual degrades gracefully, since the universal evidence layer still answers the question.

6. Remaining screens

6.1 Incidents

An incident is several alerts on one host correlated into a sequence. Render it as a horizontal kill-chain ribbon, ordered by stage rather than only by time:

```
HOST 10.4.2.19                  incident opened 14:02:11 · 3 alerts · CRITICAL

  ●──────────────●──────────────────────────●
  PS             BC                         EX
  14:02:11       14:07:44                   14:22:07
  port scan      beaconing                  exfiltration
  conf 0.88      conf 0.91                  conf 0.79
  ──────────────────────────────────────────────────────
  20 min elapsed
```

Each node opens the corresponding alert's evidence. The ribbon is the argument: these three findings are one story, and the sequence is itself evidence. This screen is where the fusion layer's work becomes visible, and it is worth a full minute of the demo.

6.2 System

Four panels. This screen exists to answer requirement (d) and to prove the read-only constraint.

- **Throughput** — flows per second, packets per second and megabits per second over time, with the sustained figure called out as a large readout and the tested peak labelled.
- **Latency** — histogram of packet-to-alert delay with median and 95th percentile marked on the axis.
- **Detector status** — eight rows: name, state, alerts produced, mean scoring time, model version. Degraded detectors show as degraded rather than disappearing.
- **Constraint panel** — egress state, ingest direction, and payload-decryption status, each as a plain readout. Three lines that say the system sent nothing, received one way only, and decrypted nothing. Point at this panel during the demo.

6.3 Host

Investigation view for one machine: a timeline of its alerts, its learned baseline against current behaviour, its JA3 history, its destination set with novel destinations marked, and every incident it appears in. Reached by clicking any IP address anywhere in the product — every IP in the interface is a link to its host view. That single interaction rule makes the whole product feel navigable.

6.4 Replay

Demo control, deliberately plain: select a capture, set speed multiplier, start and stop, and a progress readout showing position and elapsed time. One extra control earns its place — a scenario label showing which attacks the loaded capture is known to contain, read from the ground-truth manifest. Being able to say "this capture contains a beacon at minute seven, and here it is at minute seven" is a strong demo moment.

6.5 Analyst panel

A right-docked panel, 380px, toggled by keyboard or by an "Explain" control on any alert. Not a page and not a floating chat bubble.

- Opening it with an alert selected produces a plain-language explanation of that alert immediately, without the user typing anything.
- Every claim in the generated text renders with the evidence field it came from as a small inline reference. Statements that cannot be traced to a stored field must not be shown at all.
- A question box below, for queries over historical alerts.
- When the analyst service is unavailable, the panel says so plainly and the rest of the product is unaffected. This layer is never on the critical path.

7. Data contracts

The frontend team should be able to build everything above without the pipeline running. These shapes are the contract.

WebSocket — live alerts

```
{
  "type": "alert",
  "alert_id": "a7f3c210",
  "ts": 1735689612.880,
  "detected_at": 1735689612.922,
  "flow_id": "10.4.2.19:185.62.11.4:443:tcp:1735689612.880",
  "src_ip": "10.4.2.19", "dst_ip": "185.62.11.4", "dst_port": 443,
  "threat_class": "c2_beaconing",
  "confidence": 0.91,
  "severity": "critical",
  "detector": "beacon_periodicity",
  "detector_version": "0.3.1",
  "occurrences": 42,
  "first_seen": 1735688100.100,
  "evidence": [
    { "feature": "interval_cv", "value": 0.041,
      "threshold": 0.15, "direction": "below", "scale": [0, 0.5] },
    { "feature": "observed_periods", "value": 42,
      "threshold": 8, "direction": "above", "scale": [0, 50] },
    { "feature": "mean_interval_s", "value": 60.2 }
  ],
  "visual": {
    "kind": "beacon_comb",
    "timestamps": [1735688100.1, 1735688160.3, ...],
    "jitter_pct": 4.1
  },
  "mitre_technique": "T1071.001",
  "incident_id": "i2c9f004"
}
```

The `visual` object carries whatever the class-specific component needs, keyed by `kind`. The frontend switches on that key and renders the matching component. An unknown `kind` falls back to evidence bars alone, so a new detector never breaks the interface.

WebSocket — metrics, once per second

```
{ "type": "metrics", "ts": 1735689612.0,
  "flows_per_sec": 3412, "packets_per_sec": 41200, "mbps": 284.1,
  "latency_p50_ms": 18, "latency_p95_ms": 42,
  "detectors_online": 8, "detectors_total": 8,
  "egress_blocked": true, "uptime_s": 862 }
```

REST

```
GET /alerts?from=&to=&class=&severity=&host=&limit=   historical, paginated
GET /alerts/{id}                                     single alert, full evidence
GET /incidents?from=&to=                             correlated incidents
GET /incidents/{id}                                  incident with member alerts
GET /hosts/{ip}                                      baseline, history, alerts
GET /system/health                                   detector status, versions
POST /replay/start  { capture, speed }               demo control
POST /replay/stop
```

8. Stack and performance

8.1 What Node is actually for

The backend is FastAPI. React should talk to it directly — WebSocket for live alerts and metrics, REST for history. Inserting a Node proxy in front of a Python API adds a network hop and latency to the one path where latency is graded.

Node earns its place for one thing instead, and it is a valuable one: **a mock and replay server** that emits the exact WebSocket and REST contracts above from a recorded or synthetic alert file.

- The frontend team builds and polishes every screen without the pipeline existing.
- Design iteration does not require Zeek, Docker, or a capture file.
- You have a demo fallback. If the pipeline fails on stage, switch the WebSocket URL and the interface keeps running on recorded alerts. Rehearse this switch.

Recommended: React 18 with Vite, TypeScript, Tailwind with the tokens above defined as CSS variables, Zustand for state, visx or D3 for the custom visuals, and canvas for the Wire. Avoid component libraries — every one of them will fight the instrument aesthetic, and the distinctive components here are all custom anyway.

8.2 Performance rules

Under flood conditions the backend can emit alerts faster than React can render them. This is not hypothetical; it will happen during your DDoS demo, which is the worst possible moment for the interface to stall.

Rule	Why
Ring buffer, capped at 500 alerts	Memory stays flat during a flood. Older alerts remain available over REST.
Batch state updates on animation frame	Never call a state setter per message. Accumulate incoming alerts and flush once per frame.
Virtualize the alert list	Render only visible rows. Without this, 500 rows of live-updating DOM will drop frames.
Wire on canvas, outside React	The animation loop must never trigger a React render.
Memoize the evidence panel on alert id	It re-renders on every stream update otherwise.
Throttle the visible counter to 4Hz	A number changing at 60Hz is unreadable and looks like noise. The underlying count stays exact.

> **TEST THIS DELIBERATELY**
> Have the mock server replay a synthetic flood at several hundred alerts per second and confirm the interface holds sixty frames. Do this in week one, not the night before. An interface that freezes during the DDoS demo undoes every other thing the team built.

8.3 Quality floor

- Severity is never communicated by colour alone — every severity carries a text label as well as its colour, so the display survives projection and colour-vision differences.
- Keyboard navigation in the alert stream: `j` and `k` to move, `Enter` to open, `Esc` to clear. Security tools are keyboard-driven, and it makes the demo faster to drive.
- Visible focus rings, 1px `--rule-bright`, never removed.
- `prefers-reduced-motion` respected as described in section 2.5.
- The layout holds down to 1280px. Below that is out of scope — this is a wall-display product and there is no reason to spend time on phone layouts.

9. Copy

Words in this interface are readouts, not commentary. Keep them in the register of an instrument: factual, present tense, no hedging, no personality.

Write this	Not this
Waiting for first detection. Replay running at 5×.	No alerts found!
Feed stopped. Last alert 00:04:12 ago.	Oops — something went wrong
Analyst unavailable. Detection is unaffected.	Sorry, I couldn't process that request
Baseline learning. 14 minutes remaining.	Please wait…
No traffic on this interface.	Nothing to see here

Empty states name what the system is doing and what happens next. Errors state what failed and what still works. Nothing apologises — an instrument that apologises is an instrument nobody trusts.

Threat classes are always written out in full in headings and shortened to the two-letter code only in dense listings. Never mix the two in one view.

10. Build order

#	Task	Notes
1	Node mock server emitting the contracts in section 7	Everything else depends on this. Half a day, unblocks the whole team.
2	Tokens, type scale, layout shell, nav rail, instrument bar	Static, no data. Get the aesthetic right before adding motion.
3	Alert stream with virtualization and the ring buffer	Test against the flood replay immediately.
4	Evidence bars	The universal layer. Every alert renders after this.
5	The Wire	Signature element. Canvas, own animation loop.
6	Beacon comb, fan-out matrix, entropy-against-rate	The three highest-value class visuals.
7	System view	Throughput, latency, constraint panel. Graded requirement.
8	Incident ribbon	Makes the fusion layer visible.
9	Remaining class visuals, host view, replay control	Degrades gracefully if time runs short.
10	Analyst panel	Last. Never on the critical path.

> **IF ONLY THREE THINGS GET BUILT WELL**
> The alert stream, the evidence bars, and the beacon comb. Those three answer "is it detecting", "why did it fire", and "is the detection real" — which are the only three questions a judge will actually ask. Everything else in this document is amplification.